

## 5.5 Recursion Run Amok

Although recursion is a very powerful tool, it can easily be misused in various ways. In this section, we examine several cases in which a poorly implemented recursion causes drastic inefficiency, and we discuss some strategies for recognizing and avoid such pitfalls.

We begin by revisiting the *element uniqueness problem*, defined on page 174 of Section 4.3.3. We can use the following recursive formulation to determine if all  $n$  elements of a sequence are unique. As a base case, when  $n = 1$ , the elements are trivially unique. For  $n \geq 2$ , the elements are unique if and only if the first  $n - 1$  elements are unique, the last  $n - 1$  items are unique, and the first and last elements are different (as that is the only pair that was not already checked as a subcase). A recursive implementation based on this idea is given in Code Fragment 5.12, named `unique3` (to differentiate it from `unique1` and `unique2` from Chapter 4).

```

1  /** Returns true if there are no duplicate values from data[low] through data[high].*/
2  public static boolean unique3(int[] data, int low, int high) {
3      if (low >= high) return true;           // at most one item
4      else if (!unique3(data, low, high-1)) return false; // duplicate in first n-1
5      else if (!unique3(data, low+1, high)) return false; // duplicate in last n-1
6      else return (data[low] != data[high]); // do first and last differ?
7  }
```

**Code Fragment 5.12:** Recursive `unique3` for testing element uniqueness.

Unfortunately, this is a terribly inefficient use of recursion. The nonrecursive part of each call uses  $O(1)$  time, so the overall running time will be proportional to the total number of recursive invocations. To analyze the problem, we let  $n$  denote the number of entries under consideration, that is, let  $n = 1 + \text{high} - \text{low}$ .

If  $n = 1$ , then the running time of `unique3` is  $O(1)$ , since there are no recursive calls for this case. In the general case, the important observation is that a single call to `unique3` for a problem of size  $n$  may result in two recursive calls on problems of size  $n - 1$ . Those two calls with size  $n - 1$  could in turn result in four calls (two each) with a range of size  $n - 2$ , and thus eight calls with size  $n - 3$  and so on. Thus, in the worst case, the total number of method calls is given by the geometric summation

$$1 + 2 + 4 + \cdots + 2^{n-1},$$

which is equal to  $2^n - 1$  by Proposition 4.5. Thus, the running time of method `unique3` is  $O(2^n)$ . This is an incredibly inefficient method for solving the element uniqueness problem. Its inefficiency comes not from the fact that it uses recursion—it comes from the fact that it uses recursion poorly, which is something we address in Exercise C-5.12.

### An Inefficient Recursion for Computing Fibonacci Numbers

In Section 2.2.3, we introduced a process for generating the progression of Fibonacci numbers, which can be defined recursively as follows:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-2} + F_{n-1} \quad \text{for } n > 1. \end{aligned}$$

Ironically, a recursive implementation based directly on this definition results in the method `fibonacciBad` shown in Code Fragment 5.13, which computes a Fibonacci number by making two recursive calls in each non-base case.

```

1  /** Returns the nth Fibonacci number (inefficiently). */
2  public static long fibonacciBad(int n) {
3      if (n <= 1)
4          return n;
5      else
6          return fibonacciBad(n-2) + fibonacciBad(n-1);
7  }
```

**Code Fragment 5.13:** Computing the  $n^{\text{th}}$  Fibonacci number using binary recursion.

Unfortunately, such a direct implementation of the Fibonacci formula results in a terribly inefficient method. Computing the  $n^{\text{th}}$  Fibonacci number in this way requires an exponential number of calls to the method. Specifically, let  $c_n$  denote the number of calls performed in the execution of `fibonacciBad( $n$ )`. Then, we have the following values for the  $c_n$ 's:

$$\begin{aligned} c_0 &= 1 \\ c_1 &= 1 \\ c_2 &= 1 + c_0 + c_1 = 1 + 1 + 1 = 3 \\ c_3 &= 1 + c_1 + c_2 = 1 + 1 + 3 = 5 \\ c_4 &= 1 + c_2 + c_3 = 1 + 3 + 5 = 9 \\ c_5 &= 1 + c_3 + c_4 = 1 + 5 + 9 = 15 \\ c_6 &= 1 + c_4 + c_5 = 1 + 9 + 15 = 25 \\ c_7 &= 1 + c_5 + c_6 = 1 + 15 + 25 = 41 \\ c_8 &= 1 + c_6 + c_7 = 1 + 25 + 41 = 67 \end{aligned}$$

If we follow the pattern forward, we see that the number of calls more than doubles for each two consecutive indices. That is,  $c_4$  is more than twice  $c_2$ ,  $c_5$  is more than twice  $c_3$ ,  $c_6$  is more than twice  $c_4$ , and so on. Thus,  $c_n > 2^{n/2}$ , which means that `fibonacciBad( $n$ )` makes a number of calls that is exponential in  $n$ .

## An Efficient Recursion for Computing Fibonacci Numbers

We were tempted into using the bad recursive formulation because of the way the  $n^{\text{th}}$  Fibonacci number,  $F_n$ , depends on the two previous values,  $F_{n-2}$  and  $F_{n-1}$ . But notice that after computing  $F_{n-2}$ , the call to compute  $F_{n-1}$  requires its own recursive call to compute  $F_{n-2}$ , as it does not have knowledge of the value of  $F_{n-2}$  that was computed at the earlier level of recursion. That is duplicative work. Worse yet, both of those calls will need to (re)compute the value of  $F_{n-3}$ , as will the computation of  $F_{n-1}$ . This snowballing effect is what leads to the exponential running time of `fibonacciBad`.

We can compute  $F_n$  much more efficiently using a recursion in which each invocation makes only one recursive call. To do so, we need to redefine the expectations of the method. Rather than having the method return a single value, which is the  $n^{\text{th}}$  Fibonacci number, we define a recursive method that returns an array with two consecutive Fibonacci numbers  $\{F_n, F_{n-1}\}$ , using the convention  $F_{-1} = 0$ . Although it seems to be a greater burden to report two consecutive Fibonacci numbers instead of one, passing this extra information from one level of the recursion to the next makes it much easier to continue the process. (It allows us to avoid having to recompute the second value that was already known within the recursion.) An implementation based on this strategy is given in Code Fragment 5.14.

```

1  /** Returns array containing the pair of Fibonacci numbers, F(n) and F(n-1). */
2  public static long[] fibonacciGood(int n) {
3      if (n <= 1) {
4          long[] answer = {n, 0};
5          return answer;
6      } else {
7          long[] temp = fibonacciGood(n - 1);           // returns {F_{n-1}, F_{n-2}}
8          long[] answer = {temp[0] + temp[1], temp[0]}; // we want {F_n, F_{n-1}}
9          return answer;
10     }
11 }
```

**Code Fragment 5.14:** Computing the  $n^{\text{th}}$  Fibonacci number using linear recursion.

In terms of efficiency, the difference between the bad and good recursions for this problem is like night and day. The `fibonacciBad` method uses exponential time. We claim that the execution of method `fibonacciGood(n)` runs in  $O(n)$  time. Each recursive call to `fibonacciGood` decreases the argument  $n$  by 1; therefore, a recursion trace includes a series of  $n$  method calls. Because the nonrecursive work for each call uses constant time, the overall computation executes in  $O(n)$  time.

### 5.5.1 Maximum Recursive Depth in Java

Another danger in the misuse of recursion is known as *infinite recursion*. If each recursive call makes another recursive call, without ever reaching a base case, then we have an infinite series of such calls. This is a fatal error. An infinite recursion can quickly swamp computing resources, not only due to rapid use of the CPU, but because each successive call creates a frame requiring additional memory. A blatant example of an ill-formed recursion is the following:

```
1  /** Don't call this (infinite) version. */  
2  public static int fibonacci(int n) {  
3      return fibonacci(n);                // After all  $F_n$  does equal  $F_n$   
4  }
```

However, there are far more subtle errors that can lead to an infinite recursion. Revisiting our implementation of binary search (Code Fragment 5.3), when we make a recursive call on the right portion of the sequence (line 15), we specify the subarray from index `mid+1` to `high`. Had that line instead been written as

```
return binarySearch(data, target, mid, high); // sending mid, not mid+1
```

this could result in an infinite recursion. In particular, when searching a range of two elements, it becomes possible to make a recursive call on the identical range.

A programmer should ensure that each recursive call is in some way progressing toward a base case (for example, by having a parameter value that decreases with each call). To combat against infinite recursions, the designers of Java made an intentional decision to limit the overall space used to store activation frames for simultaneously active method calls. If this limit is reached, the Java Virtual Machine throws a `StackOverflowError`. (We will further discuss the “stack” data structure in Section 6.1.) The precise value of this limit depends upon the Java installation, but a typical value might allow upward of 1000 simultaneous calls.

For many applications of recursion, allowing up to 1000 nested calls suffices. For example, our `binarySearch` method (Section 5.1.3) has  $O(\log n)$  recursive depth, and so for the default recursive limit to be reached, there would need to be  $2^{1000}$  elements (far, far more than the estimated number of atoms in the universe). However, we have seen several linear recursions that have recursive depth proportional to  $n$ . Java’s limit on the recursive depth might disrupt such computations.

It is possible to reconfigure the Java Virtual Machine so that it allows for greater space to be devoted to nested method calls. This is done by setting the `-Xss` runtime option when starting Java, either as a command-line option or through the settings of an IDE. But it is often possible to rely upon the intuition of a recursive algorithm, yet to reimplement it more directly using traditional loops rather than method calls to express the necessary repetition. We discuss just such an approach to conclude the chapter.